Université de Perpignan
C. Legrand

L3 Informatique
Année 2026

TD4 Sécurité Informatique

Projet VBashX – Virus par ajout en Bash

Avertissement éthique et légal

Les techniques présentées dans ce TD sont étudiées dans un **cadre strictement académique** afin de comprendre les mécanismes utilisés par les logiciels malveillants et ainsi pouvoir mieux s'en protéger.
Toute utilisation de ces techniques en dehors du cadre pédagogique est illégale et passible de sanctions pénales (chapitre III du Code pénal, articles 323-1 à 323-8 – *Des atteintes aux systèmes de traitement automatisé de données*) :

- Accès ou maintien frauduleux dans un STAD (art. 323-1) : **3 ans** d'emprisonnement et **100 000 €** d'amende
- Entrave au fonctionnement d'un STAD (art. 323-2) : **5 ans** d'emprisonnement et **150 000 €** d'amende
- Introduction frauduleuse de données dans un STAD (art. 323-3) : **5 ans** d'emprisonnement et **150 000 €** d'amende

Peines portées à **7 ans** et **300 000 €** lorsque le système visé traite des données à caractère personnel mis en œuvre par l'État, et jusqu'à **10 ans** et **300 000 €** en bande organisée (art. 323-4-1). *Peines mises à jour par la loi LOPMI n° 2023-22 du 24 janvier 2023.*

Dans ce TD-projet, vous allez construire **brique par brique** un virus par ajout (*appending*) en Bash, baptisé **VBashX**. Chaque phase ajoute une fonctionnalité au virus de la phase précédente, en reprenant et complétant les notions vues en cours (vbashp, anti-surinfection, polymorphisme par réordonnancement de lignes, charge utile conditionnelle).

Prérequis : TD3 (scripts Bash, virus écrasant, chiffrement par substitution).

Méthode de travail : Pour chaque phase, procédez comme suit :

- Créez un répertoire de test dédié à la phase (ex : `test_phase1/`).
- Copiez-y votre virus et les fichiers cibles (`cible1.sh`, `cible2.sh`).
- Exécutez le virus et analysez les résultats **avant** de passer à la phase suivante.
- Ne travaillez **jamais** directement dans votre répertoire de développement.

Fichiers cibles de test :

```bash
#!/bin/bash
echo "I am target program 1"
echo "Running normally..."
```

cible1.sh

```bash
#!/bin/bash
echo "I am target program 2"
echo "Everything is fine."
```

cible2.sh

Exercice 1 (Projet VBashX).

Phase 1 – Virus par ajout simple

Rappel de cours

Virus par ajout (*appending virus*) :
Contrairement au virus écrasant (TD3) qui détruit le programme hôte, le virus par ajout **ajoute** son code à la fin du fichier cible. Le programme original reste fonctionnel.

Schéma textuel :

```
AVANT infection :                APRES infection :
+---------------------+         +---------------------+
| Programme hote      |         | Programme hote      |
| (code original)     |         | (code original)     |
| N lignes            |         | N lignes            |
+---------------------+         +---------------------+
                                 | Code du virus       |
                                 | (ajout en fin)      |
                                 | VIRUS_SIZE lignes   |
                                 +---------------------+

Le programme hote s'execute d'abord, puis le code viral.
```

Commandes clés :

- `tail -n K $0` : extrait les `K` dernières lignes du script en cours d'exécution (auto-extraction du code viral)
- `» fichier` : ajoute en fin de fichier (sans écraser)
- `basename $0` : nom du script sans le chemin

1a) Écrire un script `vbashx.sh` qui, pour chaque fichier `.sh` du répertoire courant (autre que lui-même), ajoute ses propres `VIRUS_SIZE` dernières lignes à la fin du fichier cible. Indication : utiliser `tail -n $VIRUS_SIZE "$0" » "$f"`.

1b) Tester : créer `cible1.sh` et `cible2.sh` (simples `echo`). Exécuter le virus, puis les cibles. Le code hôte s'exécute-t-il toujours ?

1c) Exécuter un fichier infecté. Que se passe-t-il ? Pourquoi ?

Phase 2 – Lutte contre la surinfection

Rappel de cours

Surinfection et stratégies anti-surinfection :
La **surinfection** (ou réinfection) se produit lorsqu'un virus infecte un fichier déjà infecté. Cela provoque une croissance indéfinie de la taille des fichiers, ce qui constitue un **indicateur de compromission** (IoC) facilement détectable.

Trois stratégies principales :

Stratégie	Principe	Caractéristique
Par marqueur	Insérer une signature unique dans le code viral ; vérifier sa présence avant infection	Discriminante (la signature est détectable par un antivirus)
Par comparaison tail	Comparer les dernières lignes de la cible avec celles du virus	Non discriminante mais fragile au polymorphisme
Par exécution test	Exécuter les dernières lignes avec un argument test et analyser la sortie	Non incriminante mais plus complexe

2a) **Stratégie par marqueur :** Modifier le virus pour insérer une signature (ex : `VBX_7k9Pz`) dans le code viral. Avant d'infecter un fichier, vérifier avec `grep` si la signature est déjà présente. Indication : `[ -z "$(grep "VBX_7k9Pz" "$f")" ]`.

2b) **Stratégie par comparaison tail :** Implémenter une alternative qui compare les `VIRUS_SIZE` dernières lignes de la cible avec celles du virus. Utiliser `echo -n $(tail -n ...)` pour normaliser les retours à la ligne.

2c) **Stratégie par exécution test :** Expliquer le principe (sans implémenter) : isoler les dernières lignes du fichier, les exécuter avec un argument spécial (ex : `-vbx-test`), analyser la sortie. Quels sont les avantages et inconvénients ?

2d) Remplir un tableau comparatif des 3 stratégies selon les critères : fiabilité, discrétion, robustesse au polymorphisme.

Phase 3 – Charge utile conditionnelle

Rappel de cours

Charge utile (*payload*) et déclencheurs :
La **charge utile** est l'action malveillante effectuée par le virus (affichage de message, suppression de fichiers, chiffrement, etc.). Elle est généralement activée par un **déclencheur** (*trigger*) conditionnel :

- **Temporel :** activée à une date/heure précise (ex : vendredi, 1er avril)
- **Par compteur :** activée après un certain nombre d'exécutions
- **Événementiel :** activée par un événement système (connexion réseau, fichier présent, etc.)

Commandes utiles :

- `date +%u` : jour de la semaine (1=lundi, . . . , 5=vendredi, 7=dimanche)
- `date +%d` : jour du mois (01–31)
- Fichiers cachés : un nom commençant par `.` (point) est masqué par `ls`

Lien avec TD02 : les bombes logiques utilisent le même principe de déclencheur conditionnel.

3a) Ajouter une charge utile activée uniquement le **vendredi**. Utiliser `date +%u` (5=vendredi). La charge utile sera un simple `echo` (pas destructif !). Intégrer cette fonctionnalité au virus de Phase 2a.

3b) Variante avec **compteur** : utiliser un fichier caché `.vbx_count` incrémenté à chaque exécution du virus. La charge utile se déclenche après 3 exécutions.

3c) Proposer une méthode pour tester le déclencheur vendredi sans attendre vendredi. Indication : explorer les options de `date -d`.

Phase 4 – Polymorphisme par réordonnancement de lignes

Rappel de cours

Polymorphisme par permutation de lignes :
Le **polymorphisme** consiste à faire en sorte que chaque copie du virus soit différente, rendant la détection par signature du code viral plus difficile.
Dans l'approche par **réordonnancement de lignes** (vue en cours), le code d'infection est découpé en lignes numérotées (`#@1#` à `#@LI#`). À chaque infection, ces lignes sont écrites dans un **ordre aléatoire** différent.

Structure du virus polymorphe (dans un fichier infecté) :

```
+-----------------------------------+
| Code hote original                |
+-----------------------------------+
| Code de RESTAURATION (ordre fixe) |  <-- REST_SIZE lignes
| - extrait les lignes taguees      |
| - les remet dans l'ordre          |
| - execute le code restaure        |
+-----------------------------------+
| Lignes taguees #@1# a #@LI#       |  <-- LI lignes
| (dans un ordre ALEATOIRE)         |      (en ordre permute)
+-----------------------------------+
VIRUS_SIZE = REST_SIZE + LI
```

Formule de permutation : $n_i = (r \times 3^i) \bmod 17$
Les puissances de 3 modulo 17 (nombre premier) engendrent tous les entiers de 1 à 16. Avec un germe aléatoire r, la formule itérative $r \leftarrow (r \times 3) \bmod 17$ produit une permutation complète.

Table des permutations (germe $r = 1$) :

Itération	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Ligne n_i	3	9	10	13	5	15	11	16	14	8	7	4	12	2	6	1

Processus d'exécution d'un fichier infecté :

1. Le code hôte s'exécute normalement
2. Le code de **restauration** s'exécute :
   (a) Extrait les lignes `#@1#` à `#@LI#` dans l'ordre
   (b) Les écrit dans un fichier temporaire
   (c) Exécute le fichier temporaire
3. Le fichier temporaire (code d'infection en ordre) s'exécute et infecte les cibles avec une **nouvelle permutation**

4a) Code de restauration : Écrire le code (non tagué, ordre fixe) qui extrait les lignes `#@1#` à `#@LI#` du fichier courant dans l'ordre, les écrit dans un fichier temporaire, l'exécute, puis fait `exit 0`. Le fichier temporaire doit connaître le chemin du fichier source (via une variable `SRC`).

Indication : `grep "#@${nb}#" "$0"` pour extraire la ligne numéro `nb`.

4b) **Code d'infection tagué :** Écrire le code d'infection en **16 lignes**, chacune taguée `#@n#` (le tag est un commentaire Bash en fin de ligne). Ce code doit :

- Vérifier la surinfection par marqueur
- Ajouter le code de restauration à la cible (via `tail -n $VS "$SRC" | head -n $REST_S`)
- Ajouter les 16 lignes taguées dans un **ordre aléatoire** en utilisant la formule $r \leftarrow (r \times 3) \bmod 17$ avec un germe initial aléatoire : `r=$(( ($RANDOM % 16) + 1 ))`

4c) Tester : infecter 2 cibles, comparer le contenu viral dans chacune. Les lignes taguées sont-elles dans un ordre différent ?

4d) Parmi les 3 stratégies de Phase 2, lesquelles fonctionnent encore avec le polymorphisme ? Justifier.

Phase 5 – Furtivité

Rappel de cours

Techniques de furtivité (*stealth*) :
Un virus furtif cherche à dissimuler les traces de son activité :

- **Préservation des horodatages :** la modification d'un fichier change sa date de dernière modification (`mtime`). Un administrateur surveillant les dates peut détecter l'infection.
- **Nettoyage des fichiers temporaires :** les fichiers `.vbx_tmp.sh` laissés par le virus sont des preuves.
- **Suppression des erreurs :** les messages d'erreur sur `stderr` peuvent alerter l'utilisateur.

Commandes clés :

- `cp -p fichier copie` : copie en préservant les attributs (horodatages, permissions)
- `touch -r reference fichier` : applique les horodatages de `reference` à `fichier`
- `trap "commandes" EXIT` : exécute `commandes` à la sortie du script (même en cas d'interruption Ctrl+C)
- `2>/dev/null` : redirige `stderr` vers `/dev/null` (supprime les erreurs)

5a) **Préservation des horodatages :** Avant d'infecter un fichier, sauvegarder une copie de référence avec `cp -p "$f" ".vbx_ref_$$"`, puis restaurer les horodatages après infection avec `touch -r ".vbx_ref_$$" "$f"`.

5b) **Nettoyage des fichiers temporaires :** Utiliser `trap "rm -f $TMP .vbx_ref_*" EXIT` dans le code de restauration pour garantir le nettoyage même en cas d'interruption.

5c) **Suppression des erreurs :** Ajouter `2>/dev/null` sur les commandes critiques (`grep`, `tail`, `cp`). Quelles erreurs cela masque-t-il ?

Phase 6 – Analyse et détection

Rappel de cours

Méthodologie d'analyse – Indicateurs de compromission (IoC) :
Un **indicateur de compromission** (*Indicator of Compromise*, IoC) est un artefact observable qui signale une infection. Les types de détection :

- **Par signature** (*pattern matching*) : recherche de chaînes de caractères connues (marqueurs, noms de variables)
- **Heuristique** : détection de patterns suspects (ex : un script qui se copie lui-même)
- **Comportementale** : analyse du comportement à l'exécution (ex : un script qui modifie d'autres scripts)

Commandes `grep` utiles pour l'analyse :

- `grep -q "pattern" fichier` : mode silencieux, retourne uniquement le code de sortie (0 si trouvé)
- `grep -l "pattern" *.sh` : liste les fichiers contenant le pattern
- `grep -c "pattern" fichier` : compte les occurrences

6a) Scanner : Écrire un script `scanner.sh` qui parcourt les fichiers `.sh` d'un répertoire (passé en argument ou `.` par défaut) et détecte les IoC suivants :

- Signature du marqueur : `VBX_7k9Pz`
- Marqueurs de lignes taguées : `#@n#`
- Pattern comportemental d'auto-réplication : `tail.*$0.*»`
- Pattern de restauration : `grep.*#@`

Afficher pour chaque fichier les IoC détectés, puis un résumé (infectés/sains).

6b) Tableau IoC : Pour chaque phase (1–5), identifier la technique employée et l'IoC détectable correspondant.

6c) Réflexion : Le polymorphisme par réordonnancement rend-il la détection impossible ? Quel invariant reste exploitable ? Quel type de détection (syntaxique, comportementale, heuristique) serait le plus efficace ?